

Monitoring & Observability

Simple handwritten-style interview notes - Days 73 to 77

MASTER LINE TO REMEMBER

Monitoring tells me there is a problem. Observability helps me understand the problem using metrics, logs and traces.

Memory trick: Metrics = WHAT? Logs = WHY? Traces = WHERE?

1. Monitoring vs Observability

Monitoring means watching known things: CPU, memory, disk, containers, application availability and errors.

Example: CPU is 95%. Monitoring tells me something is wrong.

Observability helps me investigate deeper: what happened, why it happened, and where the problem happened.

2. Full Architecture - Learn This Story

```
HOST -----> Node Exporter ----\
\
CONTAINERS ----> cAdvisor -----> Prometheus ----> Grafana
^
Docker Logs --> Promtail --> Loki -----|
|
Application --> OpenTelemetry Collector -----+
Metrics / Logs / Traces
```

Easy story: Node Exporter watches the machine. cAdvisor watches containers. Prometheus collects metrics. Promtail ships logs to Loki. OpenTelemetry handles telemetry. Grafana shows the information.

3. Prometheus

Prometheus is mainly used to collect and store metrics.

It normally uses a pull model: a target exposes metrics, then Prometheus scrapes those metrics again and again.

Metric = numeric measurement. Examples: CPU 35%, memory 62%, requests 1000.

Target = something Prometheus monitors.

Scrape = Prometheus collecting metrics from a target.

PromQL = Prometheus Query Language.

Useful query: up

up = 1 means target reachable. up = 0 means target is down/unreachable.

4. prometheus.yml

This file tells Prometheus WHAT to monitor.

```
global:
  scrape_interval: 15s

  scrape_configs:
    - job_name: node-exporter
      static_configs:
        - targets: ["node-exporter:9100"]

    - job_name: docker
      static_configs:
        - targets: ["cadvisor:8080"]

    - job_name: otel-collector
      static_configs:
        - targets: ["otel-collector:8889"]
```

Memory trick: docker-compose.yml = RUN services. prometheus.yml = MONITOR targets.

5. Node Exporter

Node Exporter gives HOST/MACHINE metrics.

Think: NODE = MACHINE.

It exposes CPU, RAM, disk, filesystem, network and load metrics.

Default port used in the lab: 9100.

Interview line: I use Node Exporter for host-level metrics. Prometheus scrapes those metrics and Grafana visualizes them.

6. cAdvisor

cAdvisor gives CONTAINER metrics.

It helps monitor container CPU, memory, filesystem and network usage.

Port used in the lab: 8080.

Example metric: container_memory_usage_bytes.

Memory trick: Node Exporter = machine. cAdvisor = container.

7. Grafana

Grafana is the visualization layer - think of it as the TV screen.

Prometheus collects/stores metrics. Grafana reads those metrics and displays dashboards.

Loki can also be a Grafana data source for logs.

In the lab we used panels such as CPU Usage %, Memory Usage %, Container CPU %, Container Memory MB and Docker Logs.

8. Logs - Promtail + Loki

```
Docker logs -> Promtail -> Loki -> Grafana
```

Promtail = log delivery person. It reads/collects logs and sends them to Loki.

Loki = log warehouse. It stores logs and lets us query them.

Grafana = displays the logs.

Memory trick: Promtail = SEND, Loki = STORE/QUERY, Grafana = SHOW.

9. LogQL

LogQL is Loki's query language.

Prometheus -> PromQL. Loki -> LogQL.

Show Docker logs: {job="docker"}

Show logs containing error: {job="docker"} |= "error"

10. OpenTelemetry (OTEL)

OpenTelemetry is a vendor-neutral observability framework for telemetry such as metrics, logs and traces.

The Collector flow is very important: Receiver -> Processor -> Exporter.

Receiver = data comes IN.

Processor = work on the data. We used batch processing.

Exporter = data goes OUT to another system.

OTLP ports used: 4317 = gRPC, 4318 = HTTP. Prometheus exporter in the lab used 8889.

11. Trace and Span

A trace is the complete journey of one request through the system.

A span is one individual step inside that journey.

Memory trick: TRACE = trip. SPAN = one stop.

Example: User -> Frontend -> API -> Database. The full journey is a trace; each operation is a span.

12. Alerting

Alerting means the monitoring system evaluates conditions and tells the team when something needs attention.

Flow: Metric -> Rule -> Condition -> Pending -> Firing -> Notification

Examples practiced: HighCPUUsage, HighMemoryUsage, ContainerDown, TargetDown and HighDiskUsage.

Inactive = condition is not active. Pending = condition is true but waiting. Firing = condition stayed true and alert is active.

A pending period such as 1 minute helps avoid alerts from tiny temporary spikes.

Grafana contact point tells Grafana where notifications should be sent. In the lab we created a DevOps Team contact point.

13. Docker Compose

docker-compose.yml defines the containers/services we want Docker to run and how they connect.

Final stack: Prometheus, Grafana, Node Exporter, cAdvisor, Loki, Promtail, OTEL Collector and Notes App.

```
docker compose up -d # start
docker compose ps # check
docker compose down # stop
docker compose down -v # stop + remove volumes
```

14. Important Files and Their Job

docker-compose.yml - runs the services/containers.

prometheus.yml - defines Prometheus scrape targets.

loki/loki-config.yml - configures Loki storage/server.

promtail/promtail-config.yml - tells Promtail where logs are and where to send them.

otel-collector/otel-collector-config.yml - receivers, processors, exporters and pipelines.

grafana/provisioning/datasources/datasources.yml - automatically configures data sources such as Prometheus and Loki.

grafana/provisioning/dashboards/dashboards.yml - dashboard provisioning configuration.

15. Ports to Remember for This Lab

Grafana = 3000 | Loki = 3100 | OTEL gRPC = 4317 | OTEL HTTP = 4318

cAdvisor = 8080 | OTEL Prometheus metrics = 8889 | Prometheus = 9090 | Node Exporter = 9100

Memory trick for this lab only: 30xx Grafana/Loki, 43xx OTEL, 80xx cAdvisor, 90xx Prometheus/exporter.

16. Troubleshooting Story - Interview Gold

If an application is slow, do not randomly check everything.

Use this order:

1. Dashboard - what changed?
2. Metrics - CPU, memory, container, target?
3. Logs - what error happened?
4. Trace - which service/request is slow?
5. Fix and verify - confirm metrics/logs return to normal.

Example: Grafana shows high CPU -> Prometheus confirms it -> Loki logs show errors -> trace helps locate the slow service -> fix -> verify.

17. Interview Answer - Explain Your Project

I built a complete observability stack using Docker Compose. I used Prometheus for metrics, Node Exporter for host metrics and cAdvisor for container metrics. Promtail collected Docker logs and sent them to Loki. I used OpenTelemetry Collector to receive and process telemetry such as

traces. Grafana connected to Prometheus and Loki so I could visualize CPU, memory, container metrics and logs on dashboards. I also practiced alerting for conditions such as high CPU, high memory and unavailable targets.

18. Final Memory Page

WHAT? -> Metrics

WHY? -> Logs

WHERE? -> Traces

Machine? -> Node Exporter

Container? -> cAdvisor

Metrics? -> Prometheus

Logs? -> Loki

Traces/telemetry? -> OpenTelemetry

View? -> Grafana

Prometheus query? -> PromQL

Loki query? -> LogQL

One sentence: Compose runs. Prometheus scrapes. Promtail ships. Loki stores. OTEL receives/processes/exports. Grafana shows.

Daily memory practice: Draw the architecture once from memory, say the six tool roles aloud, then explain one troubleshooting example. Do this for 5-10 minutes instead of memorizing every command.